DEVOPS FUNDAMENTALS PROJECT REPORT

Cloud-Based Linux Server Deployment with
GitHub Version Control and Automation

Student Name	NITIN MISHRA
Batch Code	A530
Platform	Amazon Web Services (AWS)
Operating System	Ubuntu Linux
Version Control	Git and GitHub
Report Date	23 October 2026

Submitted as part of the DevOps Fundamentals Mini Project

1. Project Overview

This project demonstrates a basic cloud-based Linux server deployment workflow. An Ubuntu EC2 instance is configured on AWS, a static website is deployed using Apache HTTP Server, project files are managed using Git and GitHub, and Bash scripts are used for routine server checks, Apache monitoring, and website backups.

2. Objectives

- Create and configure cloud networking and an Ubuntu EC2 server on AWS.
- Prepare the Linux server and install required tools.
- Deploy and verify a static website through Apache.
- Use Git for version control and GitHub for remote project storage.
- Create Bash automation scripts for health checks, Apache monitoring, and backups.
- Organize project documentation and screenshots for submission.

3. Tools and Technologies

Category	Tools / Technologies
Cloud	Amazon Web Services (AWS), EC2, VPC, Subnet, Internet Gateway, Route Table, Security Group
Operating System	Ubuntu Linux
Web Server	Apache HTTP Server
Version Control	Git, GitHub
Scripting	Bash / Shell scripting
Utilities	curl, vim, Linux system and log commands

4. Implementation

Task 1 — AWS Infrastructure Setup

A custom VPC and public subnet were used for the Ubuntu EC2 server. The networking setup included an Internet Gateway, route table, and security group rules to allow required access. SSH (TCP port 22) was used for administration, and HTTP (TCP port 80) was used to access the website. The exact AWS configuration should be verified against the final console screenshots.

Task 2 — Ubuntu Server Configuration

The Ubuntu server was prepared using package update and upgrade operations. Git, Apache, curl, and vim were installed for version control, website hosting, network testing, and file editing.

Typical commands used:

```
sudo apt update
sudo apt upgrade -y
sudo apt install git apache2 curl vim -y
sudo systemctl enable --now apache2
```

Task 3 — Website Deployment

A static HTML page was created with the project title, student name, batch code, date, and the message “Hosted on AWS Ubuntu Server”. Apache served the website so it could be verified through the EC2 public IPv4 address using a web browser.

Task 4 — Git and GitHub

The project repository was initialized with Git. Project files were staged and committed, a feature branch was used during development, and changes were merged into the main branch. The project was pushed to GitHub. Screenshots were organized under the screenshots directory.

Typical commands used:

```
git init
git add .
git commit -m "Initial project commit"
git branch
git switch main
git merge feature-documentation
git push origin main
```

Task 5 — Bash Automation

The project includes the following scripts. Review the actual script files before execution and ensure they have executable permissions.

health-check.sh — displays system health information such as CPU, memory, disk usage, uptime, logged-in users, and processes.

apache-monitor.sh — checks Apache service status and is intended to help monitor or restart the service when required.

backup.sh — creates timestamped compressed backups and stores them in the Backup directory.

Example commands:

```
chmod +x health-check.sh apache-monitor.sh backup.sh
./health-check.sh
./apache-monitor.sh
./backup.sh
```

5. Log Monitoring

The project contains a log-report.txt summary with sections for failed login attempts, SSH login activity, system errors, and warning messages. This file is a report template/summary; actual log evidence should only be described as collected if the corresponding commands and output were captured.

Useful commands for collecting evidence:

```
sudo grep -i "failed password" /var/log/auth.log | tail -n 20
sudo tail -n 50 /var/log/auth.log
sudo journalctl -p warning -n 50 --no-pager
sudo journalctl -p err -n 50 --no-pager
```

6. Project Structure

The repository contains README.md, index.html, the documentation directory, the scripts directory, the screenshots directory, the Bash scripts, log-report.txt, and the Backup directory with timestamped archive files.

7. Testing and Verification

- Confirm the Apache service is active using: sudo systemctl status apache2
- Open the EC2 public IPv4 address in a browser and confirm the website loads.

- Run each Bash script and inspect its output.
- Check backup archives using: ls -lh Backup/
- Confirm repository synchronization using: git status and git log --oneline.
- Verify screenshots and documentation are visible in the GitHub repository.

8. Learning Outcomes

This project provided practical experience with AWS networking concepts, Ubuntu server administration, Apache website hosting, Git branching and merging, GitHub repository management, Bash scripting, basic monitoring, backup creation, and technical documentation.

9. Conclusion

The mini project brings together foundational cloud and DevOps activities in one workflow: infrastructure setup, server preparation, web deployment, version control, automation, and documentation. Before final submission, verify that the live repository contains the latest files and that all screenshots represent the completed setup.

Submission checklist: confirm student/batch details, check the final Git status, verify the screenshots, and upload this report PDF to the GitHub repository if required by the instructor.